SOFTWARE REQUIREMENTS SPECIFICATION

Faculty and Student Achievement Portal (Institutional Achievement Portal)

---

1. Introduction

This document presents the Software Requirements Specification (SRS) for the Faculty and Student Achievement Portal, also referred to as the Institutional Achievement Portal. The specification has been derived from a technical analysis of the existing project implementation and describes the functional and non-functional requirements of the system in a structured, IEEE 830-aligned format suitable for academic submission and technical review.

The Institutional Achievement Portal is a web-based platform intended to provide a centralized mechanism through which students record extracurricular and academic achievements, faculty members verify submitted achievements, and the system automatically awards merit points that feed into a competitive institutional leaderboard.

2. Purpose

The purpose of this document is to provide a complete and unambiguous specification of the requirements of the Institutional Achievement Portal for use by project reviewers, evaluators, developers, and academic examiners. It defines what the system does, the actors that interact with it, the data it manages, and the constraints under which it operates.

At the system level, the purpose of the portal is to provide a centralized platform for students to record achievements, allow faculty to verify them, and automatically award merit points aggregated into a competitive leaderboard, addressing the absence, in many institutions, of a unified, transparent, verifiable system for tracking achievements outside standard academics.

3. Scope

The scope covers student submission, faculty verification, automated points calculation, and centralized administration of departments, categories, and users. Objectives:

- Enable students to submit achievements with proof (certificates/documents).
- Provide faculty with a dashboard to review, approve, or reject submissions.
- Maintain an automated reward points system based on achievement categories.
- Foster competition through a gamified leaderboard.

Target users: Students (UG/PG), Faculty (department-specific approvers), Administrators (system oversight and configuration).

4. Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification
BaaS	Backend-as-a-Service; used here via Supabase

Term	Definition
SPA	Single Page Application (React + React Router)
JWT	JSON Web Token, used by Supabase Auth for sessions
RLS	Row Level Security, a PostgreSQL access-control feature
CRUD	Create, Read, Update, Delete
API	Application Programming Interface (Supabase JS client / PostgREST)
UUID	Universally Unique Identifier, used as primary key format
PostgREST	REST API layer generated over PostgreSQL by Supabase
Bucket	Supabase Storage container (e.g., certificates)

5. References

- IEEE Std 830-1998, IEEE Recommended Practice for SRS
- Supabase Documentation (Auth, Database, Storage; @supabase/supabase-js)
- React 19 and React Router DOM v7 Documentation
- Bootstrap 5 Documentation
- Vite Build Tool Documentation
- Internal technical implementation documentation of the project codebase

6. Overall Description

This section describes the general factors affecting the portal and its requirements: its relationship to its environment, principal functions, user classes, and design constraints/assumptions, ahead of the detailed requirements that follow.

7. Product Perspective

The portal is a new, self-contained product, a client-side React 19 SPA served via Vite, communicating with a Supabase BaaS instance for authentication, data persistence, and file storage. Business logic (e.g., points calculation, role verification) executes client-side and is enforced through calls to Supabase's Auth, Database, and Storage services. It runs entirely in-browser with no separate server-side application logic or native client identified.

8. Product Functions

- User registration and role-based authentication (Student, Faculty, Admin)
- Achievement submission with supporting certificate/document files
- Faculty review and approval/rejection of submissions
- Automated reward-point calculation upon approval
- Administrative management of departments, categories, and announcements
- Ranked institutional leaderboard generation
- In-app notifications to students and faculty

9. User Classes and Characteristics

User Class	Characteristics
Student	Views personal dashboard, submits achievements, views leaderboard, checks notifications. Access scoped to own data/department.
Faculty	Bound to a specific department. Reviews, approves, or rejects student achievements; views department analytics. Approvals trigger point allocation.
Administrator	Global access to configuration, departments, reward rules, all user data. Manages categories and announcements.

10. Operating Environment

Frontend: React 19, Vite, React Router DOM v7, Bootstrap 5, Bootstrap Icons, Lucide React, vanilla CSS. Backend: Supabase BaaS, Supabase Auth (identity/session), Supabase-managed PostgreSQL, Supabase Storage (certificates bucket). All communication via @supabase/supabase-js. Compatible browsers: Chrome, Firefox, Safari, Edge.

11. Design Constraints

- Must use Supabase for Auth, Database, and Storage (BaaS architecture)
- Frontend constrained to React 19 + Vite SPA, routed via React Router DOM v7
- Styling constrained to Bootstrap 5 + custom vanilla CSS
- Role/route protection relies partly on client-side LocalStorage, with backend as final enforcement
- File uploads constrained to the single certificates storage bucket

12. User Documentation

Guidance is delivered through the application interface itself, including inline validation, error banners, and success confirmations. No separate standalone user manual, help center, or admin guide is identified in the source material.

13. Assumptions and Dependencies

- Assumes continuous availability of the Supabase cloud platform
- Assumes one role per user email at signup (no native multi-role accounts)
- Assumes RLS policies are configured on the backend (specific SQL policies not detailed in source)
- Assumes the certificates bucket is correctly provisioned; if not, uploads fail silently while the achievement record still submits
- Assumes users access via a modern browser with JavaScript enabled
- Depends on @supabase/supabase-js for all frontend-backend communication

14. External Interface Requirements

User Interfaces

Role-specific layouts: Layout.jsx (Student, sidebar + topbar), FacultyLayout.jsx, AdminLayout.tsx (tab-driven). Shared components: Sidebar, Topbar, ProtectedRoute. Data shown via tables (Leaderboard, Pending Approvals) and dashboard cards.

Software Interfaces

Interface	Description
Supabase Auth	signUp, signInWithPassword, signOut, getUser/getSession
Supabase DB Client	select/insert/update/delete, incl. relational queries e.g. select('*, departments(name)')
Supabase Storage	storage.from('certificates').upload / getPublicUrl
React Router DOM v7	Client-side routing and navigation

Hardware Interfaces

None beyond a browser-capable client device with network access.

Communication Interfaces

HTTPS via the Supabase JS client, invoking PostgREST (data), GoTrue-based Auth API (identity), and Storage API (files).

15. Functional Requirements

Authentication

ID	Requirement
FR-1	Register with email, password, role, name (+reg. no., department, year for students)
FR-2	Validate input, create Supabase Auth user, insert into role table
FR-3	Log in with email, password, selected role
FR-4	Authenticate via signInWithPassword; cross-verify role table
FR-5	Redirect to correct dashboard; show error on failure
FR-6	Establish JWT session; persist role in LocalStorage
FR-7	Log out, terminate session, clear local data

Student Module

ID	Requirement
FR-8	View personal dashboard
FR-9	Submit achievement (title, category, description, certificate/document)
FR-10	Upload files to certificates bucket
FR-11	Record achievement with status Pending
FR-12	Notify department faculty on submission
FR-13	Show pending achievements in student history

Faculty Module

ID	Requirement
FR-14	View department's pending achievements
FR-15	Approve/reject with optional remarks
FR-16	Update status to Approved/Rejected
FR-17	On approval: calculate points, update total_points, insert reward_points
FR-18	Notify student of outcome

Admin Module

ID	Requirement
FR-19	CRUD on departments
FR-20	CRUD on achievement categories (incl. base points)
FR-21	Broadcast announcements
FR-22	Oversee/manage users

Leaderboard

ID	Requirement
FR-23	Display leaderboard ranked by total points
FR-24	Generate via query on students, ordered by total_points DESC
FR-25	Accessible to all authenticated roles

16. Non-Functional Requirements

Category	Requirement
Performance	SPA avoids full reloads; Vite optimizes bundling
Security	Supabase Auth (bcrypt); route protection on dashboards
Reliability & Availability	Relies on Supabase cloud uptime
Scalability	Backend auto-scales via Supabase
Maintainability	Component-based React architecture
Usability	Responsive Bootstrap 5 UI, clear errors, glassmorphism aesthetic
Compatibility	Chrome, Firefox, Safari, Edge

17. Database Requirements

Table	Purpose	Keys	Columns
users	Public profile mapping	id (UUID -> auth.users)	id, email, role
departments	Institutional departments	id (UUID)	id, name

Table	Purpose	Keys	Columns
achievement_categories	Achievement types & base points	id (UUID)	id, name, base_points
students	Student metadata	user_id -> users.id; FK department_id	user_id, first_name, last_name, register_number, department_id, year, total_points
faculty	Faculty metadata	user_id -> users.id; FK department_id	user_id, first_name, last_name, department_id
admins	Admin metadata	user_id -> users.id	user_id, first_name, last_name
achievements	Student submissions	id; FK student_id, category_id	id, student_id, category_id, title, description, status, faculty_remarks, approved_by, certificate_url, document_url, created_at
reward_points	Points audit log	id; FK student_id, achievement_id	id, student_id, achievement_id, points_awarded, awarded_at
notifications	System alerts	id; FK user_id	id, user_id, title, message, is_read, created_at

18. System Architecture

```
[ Client Browser ]
      | (React SPA)
      v
[ Frontend Routing (React Router) ] <--> [ LocalStorage (Role/Token) ]
      |
      | (Supabase JS Client / REST APIs)
      v
[ Supabase BaaS ]
  |-- [ Supabase Auth (Identity & Session) ]
  |-- [ Supabase Storage (Certificates/Documents) ]
  |-- [ PostgreSQL Database (Data & Relations) ]
```

19. Data Flow Diagrams

Level 0

```
Student ----->  [ Institutional Achievement Portal ] ----> Supabase Auth
Faculty ----->  [        (React SPA)                ] ----> Supabase Database
Admin   ----->  [                                    ] ----> Supabase Storage
```

Level 1 - Achievement Submission & Approval

```
[Student] --(Data+Files)--> [Submission Process] --(Upload)--> [Storage: certificates]
   --(Insert, status=Pending)--> [achievements]
   --(Insert)--> [notifications] --> [Faculty]

[Faculty] --(Approve/Reject)--> [Approval Process] --(Update status)--> [achievements]
   --(If Approved: Insert)--> [reward_points]
   --(If Approved: Update total_points)--> [students]
   --(Insert)--> [notifications] --> [Student]
```

Level 1 - Leaderboard

```
[User] --(Request)--> [Leaderboard Query] --(SELECT ... ORDER BY total_points DESC)-->
[students]
   --(Join)--> [departments] --(Ranked List)--> [Client Display]
```

20. Use Case Descriptions

UC-1 Register Account — Actor: Student/Faculty/Admin. Pre: unregistered valid email. Flow: fill form, select role, signUp, insert into users + role table, success, redirect to login. Post: account created.

UC-2 Login — Actor: all roles. Pre: registered account. Flow: select role, enter credentials, signInWithPassword, verify role table, store token/role, redirect to dashboard. Alt: invalid credentials/role mismatch, sign out, show error.

UC-3 Submit Achievement — Actor: Student. Pre: authenticated. Flow: fill form, attach files, upload to storage, insert achievement (Pending), notify department faculty. Alt: storage misconfigured, log warning, submit text record anyway.

UC-4 Approve/Reject Achievement — Actor: Faculty. Pre: pending achievement exists in department. Flow: select achievement, approve/reject (+remarks), update status; if approved, calculate points, update total_points, insert reward_points, notify student.

UC-5 Manage Departments/Categories — Actor: Admin. Pre: authenticated. Flow: navigate to tab, enter/edit details, insert/update/delete, refresh state.

UC-6 View Leaderboard — Actor: all roles. Flow: navigate to leaderboard, query students joined departments ordered by total_points DESC, render ranked list.

21. Activity Diagrams (Text)

Signup: Fill form -> select role -> submit -> create auth user -> insert users -> insert role table -> success message -> redirect to /login.

Login: Select role -> enter credentials -> submit -> authenticate -> valid? No: error / Yes: query role table -> found? No: sign out, Access Denied / Yes: store token+role -> redirect to dashboard.

Achievement Submission & Approval: Student fills form, attaches files -> submit -> upload files -> insert achievement (Pending) -> notify faculty -> faculty reviews -> approve/reject -> if approve: update status, calculate points, update total_points, insert reward_points, notify student / if reject: update status, notify student.

22. Sequence Diagrams (Text)

Login: User to LoginForm (submit) -> LoginForm to SupabaseAuth (signInWithPassword) -> LoginForm to SupabaseDB (role lookup) -> if found: store token/role, navigate to dashboard; else: sign out, show Access Denied.

Achievement Submission: Student to Form (fill+submit) -> Form to Storage (upload, getPublicUrl) -> Form to DB (achievements.insert) -> Form to DB (fetch matching faculty) -> Form to DB (notifications.insert) -> Form to Student (success).

Faculty Approval: Faculty to Panel (Approve) -> Panel to DB (achievements.update) -> Panel to DB (reward_points.insert) -> Panel to DB (students.update total_points) -> Panel to DB (notifications.insert) -> Panel to Faculty (refresh tables).

23. Security Requirements

- JWT-based session management via Supabase Auth
- Client-side route protection via ProtectedRoute.jsx, checking getSession() + LocalStorage userRole
- RLS expected on the Supabase backend (specific policies not detailed in source)
- Certificate/document files stored in certificates bucket with generated public URLs
- Passwords handled via Supabase Auth's bcrypt hashing, never processed in plaintext by the app

24. Performance Requirements

- SPA architecture avoids full-page reloads
- Vite optimizes asset bundling for faster load
- Backend read/write handled via Supabase PostgREST, auto-scaled by the platform

25. Testing Requirements

Features to Test

- Auth workflows for all 3 roles
- Protected-route enforcement across roles
- Certificate/document upload
- Faculty approval logic (points added only on approval, not rejection)
- Relational deletion (e.g., category linked to achievements)

Testing Methods

- Unit testing for point-calculation utilities
- Integration testing for Supabase API interactions
- End-to-end testing (Cypress/Playwright) for UI workflows

26. Future Scope

- Multi-Factor Authentication for admin accounts
- Email notifications via Supabase Edge Functions + Resend/SendGrid
- Full PDF export for student portfolios (currently scaffolded in pdfExport.js)
- Pagination on Leaderboard/Approval tables
- Soft deletes for achievements to preserve audit history

27. Appendix

Known Limitations

- One role per email; no native multi-role accounts
- Silent upload failure if storage bucket unconfigured (achievement text still submits)

- Client-side LocalStorage trusted for UX speed; backend enforces final access
- No optimistic locking for concurrent faculty review of the same achievement

Route Reference

Category	Routes
Public	/, /login, /signup
Student	/dashboard, /student-dashboard, /my-achievements, /add-achievement, /achievement/:id, /reward-points, /leaderboard, /student-search, /notifications, /profile/:id, /settings
Faculty	/faculty-dashboard, /faculty-approvals, /faculty-verified, /faculty-students, /faculty-analytics, /faculty-reports, /faculty-notifications, /faculty-profile
Admin	/admin-dashboard, /admin-users, /admin-students, /admin-faculty, /admin-reports, /admin-verifications, /admin-reward-rules, /admin-departments, /admin-categories, /admin-notifications, /admin-settings, /admin-announcements

Business Logic Summary

Workflow	Summary
Signup	Create auth user -> insert users -> insert role table
Login	Authenticate -> cross-check role table -> set LocalStorage -> redirect
Achievement	Submit (Pending) -> faculty approves -> status Approved, total_points incremented, reward_points logged, notification sent
Leaderboard	Ranks students by aggregated total_points